ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್ ಕಲಿಯಲು ಬೇಕಾದ ಗಣಿತ

ಸರಳ ಕನ್ನಡದಲ್ಲಿ ಉದಾಹರಣೆಗಳೊಂದಿಗೆ

(Maths for Neural Networks — Kannada Guide)

ಪ್ರತಿ ಪರಿಕಲ್ಪನೆಗೂ: ಅದು ಏನು → ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್‌ನಲ್ಲಿ ಎಲ್ಲಿ ಬರುತ್ತದೆ → ಲೆಕ್ಕ ಹಾಕಿದ ಉದಾಹರಣೆ

೧. ಮೊದಲು ಒಂದು ನಕ್ಷೆ: ಯಾವ ಗಣಿತ, ಎಲ್ಲಿ ಬೇಕು?

ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್ ಕಲಿಯಲು ದೊಡ್ಡ ಗಣಿತ ಪಂಡಿತರಾಗಬೇಕಿಲ್ಲ. ಕೇವಲ ಈ ಕೆಳಗಿನ ಆರು ವಿಷಯಗಳ ಮೂಲಭೂತ ತಿಳುವಳಿಕೆ ಸಾಕು. ಇವು ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್‌ನ ಮೂರು ಹಂತಗಳಿಗೆ ಹೊಂದುತ್ತವೆ:

ಹಂತ	ಏನಾಗುತ್ತದೆ?	ಬೇಕಾದ ಗಣಿತ
೧. ಮುಂದಕ್ಕೆ ಲೆಕ್ಕ (Forward pass)	ಇನ್‌ಪುಟ್ ತೆಗೆದುಕೊಂಡು ಉತ್ತರ (ಭವಿಷ್ಯ) ಕೊಡುತ್ತದೆ	ಬೀಜಗಣಿತ, ಸದಿಶ (vector), ಮ್ಯಾಟ್ರಿಕ್ಸ್, ಘಾತಾಂಕ/ಲಾಗ್
೨. ತಪ್ಪು ಅಳೆಯುವುದು (Loss)	ಉತ್ತರ ಎಷ್ಟು ತಪ್ಪು ಎಂದು ಒಂದು ಸಂಖ್ಯೆಯಲ್ಲಿ ಹೇಳುತ್ತದೆ	ಸಂಭವನೀಯತೆ, ಅಂಕಿಅಂಶ, ಲಾಗರಿದಮ್
೩. ತಿದ್ದಿಕೊಳ್ಳುವುದು (Backprop + Gradient Descent)	ತಪ್ಪು ಕಡಿಮೆ ಆಗುವಂತೆ ತೂಕಗಳನ್ನು (weights) ಬದಲಿಸುತ್ತದೆ	ವ್ಯುತ್ಪನ್ನ (derivative), ಭಾಗಶಃ ವ್ಯುತ್ಪನ್ನ, ಗ್ರೇಡಿಯಂಟ್, ಚೈನ್ ರೂಲ್

> **ಕಲಿಯುವ ಸರಿಯಾದ ಕ್ರಮ**
>
> ಬೀಜಗಣಿತ → ಸದಿಶ ಮತ್ತು ಮ್ಯಾಟ್ರಿಕ್ಸ್ → ಘಾತಾಂಕ ಮತ್ತು ಲಾಗ್ → ಸಂಭವನೀಯತೆ ಮತ್ತು ಅಂಕಿಅಂಶ → ವ್ಯುತ್ಪನ್ನ → ಗ್ರೇಡಿಯಂಟ್ ಮತ್ತು ಚೈನ್ ರೂಲ್ → ಎಲ್ಲವನ್ನೂ ಸೇರಿಸಿ ಒಂದು ನ್ಯೂರಾನ್ ತರಬೇತಿ.

೨. ಬೀಜಗಣಿತ: ಚರಾಂಕ ಮತ್ತು ಸರಳ ರೇಖೆ

ಇದು ಏನು?

ಒಂದು ಸಂಖ್ಯೆಯ ಬದಲು ಅಕ್ಷರ ಬಳಸುವುದು ಬೀಜಗಣಿತ. ಸರಳ ರೇಖೆಯ ಸೂತ್ರ `y = w·x + b` ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್‌ನ ಅತ್ಯಂತ ಮುಖ್ಯ ಸೂತ್ರ. ಇಲ್ಲಿ:

- **x** = ಇನ್‌ಪುಟ್ (ನಾವು ಕೊಡುವ ಮಾಹಿತಿ)
- **w** = ತೂಕ (weight) — ಆ ಇನ್‌ಪುಟ್ ಎಷ್ಟು ಮುಖ್ಯ ಎಂದು ಹೇಳುವ ಸಂಖ್ಯೆ
- **b** = ಬಯಾಸ್ (bias) — ಇನ್‌ಪುಟ್ ಸೊನ್ನೆ ಇದ್ದರೂ ಇರುವ ಮೂಲ ಮೌಲ್ಯ
- **y** = ಔಟ್‌ಪುಟ್ (ಉತ್ತರ)

ಉದಾಹರಣೆ: ಆಟೋ ಚಾರ್ಜ್

ಆಟೋ ಹತ್ತಿದ ತಕ್ಷಣ ₹30 ಮೀಟರ್ ಚಾರ್ಜ್ ಇರುತ್ತದೆ. ನಂತರ ಪ್ರತಿ ಕಿಲೋಮೀಟರ್‌ಗೆ ₹15. ಆಗ ಒಟ್ಟು ಬಾಡಿಗೆ:

```
ಬಾಡಿಗೆ = 15 × (ಕಿಮೀ) + 30        →   y = w·x + b   (w = 15, b = 30)
```

೪ ಕಿಮೀ ಹೋದರೆ: 15 × 4 + 30 = **₹90**.

ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್ ಮಾಡುವುದು ಇದನ್ನೇ ಉಲ್ಟಾ: ಕೆಲವು ಟ್ರಿಪ್‌ಗಳ ಕಿಮೀ ಮತ್ತು ಬಾಡಿಗೆ ನೋಡಿ, **w ಮತ್ತು b ಯಾವುವು ಎಂದು ತಾನೇ ಕಂಡುಹಿಡಿಯುತ್ತದೆ**. ಅದನ್ನೇ "ಕಲಿಯುವುದು" ಎನ್ನುತ್ತೇವೆ.

೩. ಸದಿಶ (Vector) ಮತ್ತು ಡಾಟ್ ಪ್ರಾಡಕ್ಟ್

ಇದು ಏನು?

ಸದಿಶ ಎಂದರೆ ಸಂಖ್ಯೆಗಳ ಒಂದು ಸಾಲು. ಉದಾಹರಣೆಗೆ ಒಬ್ಬ ವಿದ್ಯಾರ್ಥಿಯ ಮಾಹಿತಿ: `x = [4, 6]` (4 ಗಂಟೆ ಓದು, 6 ಗಂಟೆ ನಿದ್ದೆ). ಹಲವು ಇನ್‌ಪುಟ್‌ಗಳಿದ್ದಾಗ ಪ್ರತಿಯೊಂದಕ್ಕೂ ಒಂದು ತೂಕ ಇರುತ್ತದೆ; ಅವನ್ನೂ ಸದಿಶ ಆಗಿ ಬರೆಯುತ್ತೇವೆ: `w = [0.5, 0.2]`.

ಡಾಟ್ ಪ್ರಾಡಕ್ಟ್: ಎರಡು ಸದಿಶಗಳ ಜೋಡಿ ಜೋಡಿ ಸಂಖ್ಯೆಗಳನ್ನು ಗುಣಿಸಿ, ಎಲ್ಲವನ್ನೂ ಕೂಡುವುದು.

```
x · w = (4 × 0.5) + (6 × 0.2) = 2 + 1.2 = 3.2
```

ಉದಾಹರಣೆ: ಪರೀಕ್ಷೆಯಲ್ಲಿ ಪಾಸ್ ಆಗುವ ಸಾಧ್ಯತೆ

ಒಂದು ನ್ಯೂರಾನ್ ಹೀಗೆ ಲೆಕ್ಕ ಹಾಕುತ್ತದೆ: `z = w · x + b`

ಓದಿದ ಗಂಟೆ = 4, ನಿದ್ದೆ ಗಂಟೆ = 6, w = [0.5, 0.2], b = −1.5 ಆದರೆ:

```
z = (0.5×4) + (0.2×6) + (−1.5)
  = 2 + 1.2 − 1.5
  = 1.7
```

ಈ **z = 1.7** ಅನ್ನು ನಂತರ activation function ಗೆ ಕೊಡುತ್ತೇವೆ (ಭಾಗ ೧೦ ನೋಡಿ).

ನೆನಪಿಡಿ

ಒಂದು ನ್ಯೂರಾನ್ ಮಾಡುವ ಮುಖ್ಯ ಕೆಲಸ = ಡಾಟ್ ಪ್ರಾಡಕ್ಟ್ + ಬಯಾಸ್. ಅಷ್ಟೇ!

೪. ಮ್ಯಾಟ್ರಿಕ್ಸ್ ಮತ್ತು ಮ್ಯಾಟ್ರಿಕ್ಸ್ ಗುಣಾಕಾರ

ಇದು ಏನು?

ಮ್ಯಾಟ್ರಿಕ್ಸ್ ಎಂದರೆ ಸಂಖ್ಯೆಗಳ ಕೋಷ್ಟಕ (ಸಾಲು ಮತ್ತು ಕಾಲಮ್). ಒಂದು ಪದರದಲ್ಲಿ (layer) ಹತ್ತಾರು ನ್ಯೂರಾನ್‌ಗಳಿರುತ್ತವೆ. ಪ್ರತಿ ನ್ಯೂರಾನ್‌ನ ಡಾಟ್ ಪ್ರಾಡಕ್ಟ್ ಒಂದೊಂದಾಗಿ ಮಾಡುವ ಬದಲು, ಎಲ್ಲವನ್ನೂ ಒಂದೇ ಬಾರಿಗೆ **ಮ್ಯಾಟ್ರಿಕ್ಸ್ ಗುಣಾಕಾರ**ದಿಂದ ಮಾಡುತ್ತೇವೆ. ಕಂಪ್ಯೂಟರ್/GPU ಇದರಲ್ಲಿ ಬಹಳ ವೇಗ.

ಉದಾಹರಣೆ: 2 ಇನ್‌ಪುಟ್, 3 ನ್ಯೂರಾನ್‌ಗಳ ಪದರ

ಇನ್‌ಪುಟ್ `x = [1, 2]`. ತೂಕಗಳ ಮ್ಯಾಟ್ರಿಕ್ಸ್ W (2 ಸಾಲು × 3 ಕಾಲಮ್) — ಪ್ರತಿ ಕಾಲಮ್ ಒಂದು ನ್ಯೂರಾನ್‌ನ ತೂಕಗಳು:

	ನ್ಯೂರಾನ್ 1	ನ್ಯೂರಾನ್ 2	ನ್ಯೂರಾನ್ 3
x_1 ಗೆ ತೂಕ	1	0	−1
x_2 ಗೆ ತೂಕ	2	1	0

ಪ್ರತಿ ನ್ಯೂರಾನ್‌ಗೆ x · (ಅದರ ಕಾಲಮ್):

```
ನ್ಯೂರಾನ್ 1 = 1×1 + 2×2 =  5
ನ್ಯೂರಾನ್ 2 = 1×0 + 2×1 =  2
ನ್ಯೂರಾನ್ 3 = 1×(−1) + 2×0 = −1        →   x·W = [5, 2, −1]
```

ಬಯಾಸ್ `b = [0, 1, 1]` ಕೂಡಿದರೆ: `[5, 3, 0]`. ಇದೇ ಆ ಪದರದ ಔಟ್‌ಪುಟ್ (activation ಹಾಕುವ ಮೊದಲು).

ಗಮನಿಸಿ

ಮ್ಯಾಟ್ರಿಕ್ಸ್ ಗುಣಾಕಾರ ಸಾಧ್ಯವಾಗಲು ಮೊದಲ ಮ್ಯಾಟ್ರಿಕ್ಸ್‌ನ ಕಾಲಮ್‌ಗಳ ಸಂಖ್ಯೆ = ಎರಡನೆಯದರ ಸಾಲುಗಳ ಸಂಖ್ಯೆ ಇರಬೇಕು. ಇಲ್ಲಿ (1×**2**) ಮತ್ತು (**2**×3) → ಫಲಿತಾಂಶ (1×3). "shape mismatch" ದೋಷ ಬಂದರೆ ಇಲ್ಲೇ ತಪ್ಪಿರುತ್ತದೆ.

೫. ಘಾತಾಂಕ (Exponent) ಮತ್ತು ಲಾಗರಿದಮ್ (Log)

ಇದು ಏನು?

ಘಾತಾಂಕ: ಒಂದು ಸಂಖ್ಯೆಯನ್ನು ತನ್ನಿಂದ ತಾನೇ ಹಲವು ಬಾರಿ ಗುಣಿಸುವುದು ($2^3 = 8$). ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್‌ನಲ್ಲಿ ವಿಶೇಷ ಸಂಖ್ಯೆ **e ≈ 2.718** ಬಳಸಿ e^x ಬರೆಯುತ್ತೇವೆ. ಇದು ಯಾವಾಗಲೂ ಧನಾತ್ಮಕ (positive) ಸಂಖ್ಯೆಯನ್ನೇ ಕೊಡುತ್ತದೆ.

ಲಾಗರಿದಮ್: ಘಾತಾಂಕದ ಉಲ್ಟಾ. "e ಯನ್ನು ಎಷ್ಟನೇ ಘಾತಕ್ಕೆ ಏರಿಸಿದರೆ ಈ ಸಂಖ್ಯೆ ಸಿಗುತ್ತದೆ?" ಎಂದು ಕೇಳುವುದು. $\ln(e^2) = 2$.

ಸಂಖ್ಯೆ	ಬೆಲೆ	ಎಲ್ಲಿ ಬರುತ್ತದೆ
e^0	1	Sigmoid ನಲ್ಲಿ ಮಧ್ಯಬಿಂದು
e^1	2.718	Softmax ಲೆಕ್ಕ
e^{-1}	0.368	Sigmoid (ಋಣ ಇನ್‌ಪುಟ್‌ಗೆ ಚಿಕ್ಕ ಬೆಲೆ)
ln(1)	0	Loss: ಸಂಪೂರ್ಣ ಸರಿ = ತಪ್ಪು 0
ln(0.5)	−0.693	Cross-entropy ಲೆಕ್ಕ

ಉದಾಹರಣೆ: ಸಿಗ್ಮಾಯ್ಡ್ (Sigmoid)

Sigmoid ಯಾವುದೇ ಸಂಖ್ಯೆಯನ್ನು 0 ಮತ್ತು 1 ರ ನಡುವಿನ ಸಂಖ್ಯೆಯಾಗಿ ಬದಲಿಸುತ್ತದೆ:

```
σ(z) = 1 / (1 + e^(−z))
```

ಭಾಗ ೩ ರ ನಮ್ಮ z = 1.7 ಗೆ: e^(−1.7) = 0.183, ಆದ್ದರಿಂದ

```
σ(1.7) = 1 / (1 + 0.183) = 0.8455    (ಅಂದರೆ ಸುಮಾರು 85% ಪಾಸ್ ಆಗುವ ಸಾಧ್ಯತೆ)
```

z = 0 ಆದರೆ σ = 0.5 (ಎರಡೂ ಸಮ). z ತುಂಬ ದೊಡ್ಡದಾದರೆ 1 ಕ್ಕೆ, ತುಂಬ ಚಿಕ್ಕದಾದರೆ (ಋಣ) 0 ಕ್ಕೆ ಹತ್ತಿರ.

೬. ಸಂಭವನೀಯತೆ (Probability) ಮತ್ತು Softmax

ಇದು ಏನು?

ಸಂಭವನೀಯತೆ ಎಂದರೆ ಒಂದು ಘಟನೆ ನಡೆಯುವ ಸಾಧ್ಯತೆ: 0 (ಖಂಡಿತ ಇಲ್ಲ) ರಿಂದ 1 (ಖಂಡಿತ ಹೌದು). ಎಲ್ಲ ಸಾಧ್ಯತೆಗಳ ಮೊತ್ತ ಯಾವಾಗಲೂ **1**. ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್ ಒಂದು ಚಿತ್ರವನ್ನು "ಬೆಕ್ಕು / ನಾಯಿ / ಹಸು" ಎಂದು ಗುರುತಿಸುವಾಗ ಪ್ರತಿ ವರ್ಗಕ್ಕೂ ಒಂದು ಸಂಭವನೀಯತೆ ಕೊಡಬೇಕು. ಇದನ್ನು ಮಾಡುವ ಸೂತ್ರವೇ **Softmax**.

ಉದಾಹರಣೆ: ಮೂರು ವರ್ಗಗಳ Softmax

ನೆಟ್‌ವರ್ಕ್‌ನ ಕಚ್ಚಾ ಅಂಕಗಳು (logits): ಬೆಕ್ಕು = 2.0, ನಾಯಿ = 1.0, ಹಸು = 0.1

```
ಹಂತ ೧:  e^2.0 = 7.389,  e^1.0 = 2.718,  e^0.1 = 1.105
ಹಂತ ೨:  ಮೊತ್ತ = 7.389 + 2.718 + 1.105 = 11.212
ಹಂತ ೩:  ಬೆಕ್ಕು = 7.389 / 11.212 = 0.659
         ನಾಯಿ  = 2.718 / 11.212 = 0.242
         ಹಸು   = 1.105 / 11.212 = 0.099
```

ಮೂರೂ ಕೂಡಿದರೆ 0.659 + 0.242 + 0.099 = **1.000**. ಅಂದರೆ ನೆಟ್‌ವರ್ಕ್ ಪ್ರಕಾರ ಇದು 66% ಬೆಕ್ಕು, 24% ನಾಯಿ, 10% ಹಸು.

೭. ಅಂಕಿಅಂಶ (Statistics): ಸರಾಸರಿ, ವ್ಯತ್ಯಾಸ, Normalization

ಇದು ಏನು?

- **ಸರಾಸರಿ (Mean, μ)**: ಎಲ್ಲವನ್ನೂ ಕೂಡಿ ಸಂಖ್ಯೆಯಿಂದ ಭಾಗಿಸುವುದು.
- **ವ್ಯತ್ಯಾಸ (Variance)**: ಸಂಖ್ಯೆಗಳು ಸರಾಸರಿಯಿಂದ ಎಷ್ಟು ದೂರ ಹರಡಿವೆ.
- **ಪ್ರಮಾಣಿತ ವಿಚಲನೆ (Standard deviation, σ)**: ವ್ಯತ್ಯಾಸದ ವರ್ಗಮೂಲ.

ಮಾಹಿತಿಯನ್ನು ನೆಟ್‌ವರ್ಕ್‌ಗೆ ಕೊಡುವ ಮೊದಲು ಎಲ್ಲ ಇನ್‌ಪುಟ್‌ಗಳು ಒಂದೇ ಪ್ರಮಾಣದಲ್ಲಿರಬೇಕು. ಇಲ್ಲದಿದ್ದರೆ (ಉದಾ: ವಯಸ್ಸು 0–100, ಸಂಬಳ 0–1,00,000) ದೊಡ್ಡ ಸಂಖ್ಯೆ ಉಳಿದವನ್ನು ಮುಳುಗಿಸಿ ಕಲಿಕೆ ನಿಧಾನವಾಗುತ್ತದೆ. ಇದಕ್ಕೆ **normalization** ಮಾಡುತ್ತೇವೆ.

ಉದಾಹರಣೆ: ಮೂವರು ವಿದ್ಯಾರ್ಥಿಗಳ ಅಂಕಗಳು 40, 60, 80

```
ಸರಾಸರಿ μ = (40 + 60 + 80) / 3 = 60
ವ್ಯತ್ಯಾಸ  = ((40−60)² + (60−60)² + (80−60)²) / 3 = (400 + 0 + 400) / 3 = 266.7
σ         = √266.7 = 16.33

Z-score = (ಅಂಕ − μ) / σ
  40 → (40−60)/16.33 = −1.22
  60 → 0
  80 → +1.22
```

ಈಗ ಸಂಖ್ಯೆಗಳು ಸರಾಸರಿ 0, ಹರಡುವಿಕೆ ಸುಮಾರು 1 ಇರುವಂತೆ ಬದಲಾದವು — ನೆಟ್‌ವರ್ಕ್‌ಗೆ ಇದು ಕಲಿಯಲು ಅನುಕೂಲ.

ಇನ್ನೊಂದು ವಿಧಾನ **Min-Max**: (ಅಂಕ − ಕನಿಷ್ಠ)/(ಗರಿಷ್ಠ − ಕನಿಷ್ಠ) → 40→0, 60→0.5, 80→1.

೮. ವ್ಯುತ್ಪನ್ನ (Derivative): ಇಳಿಜಾರು

ಇದು ಏನು?

ವ್ಯುತ್ಪನ್ನ ಎಂದರೆ "**x ಸ್ವಲ್ಪ ಬದಲಾದರೆ y ಎಷ್ಟು ಬದಲಾಗುತ್ತದೆ?**" ಎಂಬ ಪ್ರಶ್ನೆಯ ಉತ್ತರ. ಅದು ಆ ಬಿಂದುವಿನಲ್ಲಿ ಗ್ರಾಫ್‌ನ **ಇಳಿಜಾರು (slope)**.

> **ಉದಾಹರಣೆ: ಬೆಟ್ಟದ ಇಳಿಜಾರು**
>
> ನೀವು ನಂದಿ ಬೆಟ್ಟ ಹತ್ತುತ್ತಿದ್ದೀರಿ ಎಂದುಕೊಳ್ಳಿ. ಕಾಲ್ಬದಿ ಇಳಿಜಾರು ಹೆಚ್ಚಿದ್ದರೆ ಎತ್ತರ ಬೇಗ ಬದಲಾಗುತ್ತದೆ; ಸಮತಟ್ಟಾಗಿದ್ದರೆ ಬದಲಾಗುವುದಿಲ್ಲ (ಇಳಿಜಾರು 0). ವ್ಯುತ್ಪನ್ನ ಇದೇ ಇಳಿಜಾರು.

ಫಂಕ್ಷನ್ f(x)	ವ್ಯುತ್ಪನ್ನ f′(x)	ಉದಾಹರಣೆ
ಸ್ಥಿರ ಸಂಖ್ಯೆ (5)	0	ಬದಲಾಗುವುದಿಲ್ಲ
w·x (x ಬಗ್ಗೆ)	w	ಆಟೋ: ಕಿಮೀಗೆ ₹15 → ಇಳಿಜಾರು 15
x^2	2x	x = 2 ರಲ್ಲಿ ಇಳಿಜಾರು 4
e^x	e^x	ತನ್ನದೇ ವ್ಯುತ್ಪನ್ನ
ln(x)	1/x	x = 2 ರಲ್ಲಿ 0.5
Sigmoid σ(z)	σ(z)·(1 − σ(z))	σ(1.7)=0.8455 → 0.8455×0.1545 = 0.1306
ReLU: max(0, z)	z > 0 ಆದರೆ 1, ಇಲ್ಲದಿದ್ದರೆ 0	ಸರಳ ಮತ್ತು ವೇಗ

> **ಇದು ಏಕೆ ಮುಖ್ಯ?**
>
> ಇಳಿಜಾರು ಧನಾತ್ಮಕ ಇದ್ದರೆ "w ಹೆಚ್ಚಿಸಿದರೆ ತಪ್ಪು ಹೆಚ್ಚುತ್ತದೆ" ಎಂದರ್ಥ, ಋಣ ಇದ್ದರೆ "w ಹೆಚ್ಚಿಸಿದರೆ ತಪ್ಪು ಕಡಿಮೆ ಆಗುತ್ತದೆ". ಇದರಿಂದ ಯಾವ ದಿಕ್ಕಿಗೆ ತೂಕ ಬದಲಿಸಬೇಕು ಎಂದು ತಿಳಿಯುತ್ತದೆ.

೯. ಭಾಗಶಃ ವ್ಯುತ್ಪನ್ನ ಮತ್ತು ಗ್ರೇಡಿಯಂಟ್ (Gradient)

ಇದು ಏನು?

ನೆಟ್‌ವರ್ಕ್‌ನಲ್ಲಿ ಒಂದೇ ಅಲ್ಲ, ಸಾವಿರಾರು ತೂಕಗಳಿರುತ್ತವೆ. **ಭಾಗಶಃ ವ್ಯುತ್ಪನ್ನ** ಎಂದರೆ "ಉಳಿದೆಲ್ಲವನ್ನೂ ಹಾಗೆಯೇ ಇಟ್ಟು, ಕೇವಲ ಒಂದು ತೂಕವನ್ನು ಸ್ವಲ್ಪ ಬದಲಿಸಿದರೆ loss ಎಷ್ಟು ಬದಲಾಗುತ್ತದೆ?". ಎಲ್ಲ ತೂಕಗಳ ಭಾಗಶಃ ವ್ಯುತ್ಪನ್ನಗಳನ್ನು ಒಂದು ಸದಿಶದಲ್ಲಿ ಜೋಡಿಸಿದರೆ ಅದೇ **ಗ್ರೇಡಿಯಂಟ್ (∇L)**.

ಉದಾಹರಣೆ: $L(w, b) = (w + 2b - 5)^2$, $w = 1, b = 1$

ಒಳಗಿನ ಭಾಗ $u = w + 2b - 5 = 1 + 2 - 5 = -2$, $L = u^2 = 4$

$\partial L/\partial w = 2u \times (\partial u/\partial w) = 2(-2) \times 1 = -4$
$\partial L/\partial b = 2u \times (\partial u/\partial b) = 2(-2) \times 2 = -8$

ಗ್ರೇಡಿಯಂಟ್ $\nabla L = [-4, -8]$

ಅರ್ಥ: w ಮತ್ತು b ಎರಡನ್ನೂ ಹೆಚ್ಚಿಸಿದರೆ loss ಇಳಿಯುತ್ತದೆ; b ಬದಲಾವಣೆ w ಗಿಂತ ಎರಡು ಪಟ್ಟು ಪರಿಣಾಮಕಾರಿ.

ಗ್ರೇಡಿಯಂಟ್ ಎಂದರೆ

ಕತ್ತಲೆಯಲ್ಲಿ ಬೆಟ್ಟದ ಮೇಲೆ ನಿಂತಾಗ ಕಾಲಿನ ಕೆಳಗಿನ ನೆಲದ ಓರೆ. ಅದು ಯಾವ ದಿಕ್ಕಿಗೆ ಹತ್ತುವುದು ಅತಿ ವೇಗ ಎಂದು ತೋರಿಸುತ್ತದೆ. ಇಳಿಯಬೇಕಾದರೆ ಅದರ **ವಿರುದ್ಧ ದಿಕ್ಕಿಗೆ** ಹೆಜ್ಜೆ ಇಡಬೇಕು.

೧೦. Activation Function — ಏಕೆ ಬೇಕು?

ನ್ಯೂರಾನ್ `z = w·x + b` ಲೆಕ್ಕ ಹಾಕಿದ ನಂತರ ಅದನ್ನು ಒಂದು ವಕ್ರ (non-linear) ಫಂಕ್ಷನ್ ಮೂಲಕ ಹಾಯಿಸುತ್ತೇವೆ. ಇದಿಲ್ಲದಿದ್ದರೆ ನೂರು ಪದರಗಳಿದ್ದರೂ ಅದು ಒಂದೇ ಸರಳ ರೇಖೆಯ ಸೂತ್ರಕ್ಕೆ ಸಮ ಆಗುತ್ತದೆ; ಸಂಕೀರ್ಣ ಮಾದರಿ ಕಲಿಯಲು ಸಾಧ್ಯವಿಲ್ಲ.

ಹೆಸರು	ಸೂತ್ರ	ಔಟ್‌ಪುಟ್ ವ್ಯಾಪ್ತಿ	z = 1.7 ಗೆ	z = −1.5 ಗೆ
ReLU	max(0, z)	0 ರಿಂದ ∞	1.7	0
Sigmoid	1/(1+e^−z)	0 ರಿಂದ 1	0.846	0.182
Tanh	(e^z − e^−z)/(e^z + e^−z)	−1 ರಿಂದ 1	0.935	−0.905

ಎರಡು ಅಥವಾ ಹೆಚ್ಚು ವರ್ಗಗಳ ಕೊನೆಯ ಪದರದಲ್ಲಿ Softmax (ಭಾಗ ೬) ಬಳಸುತ್ತೇವೆ. ಮಧ್ಯದ ಪದರಗಳಲ್ಲಿ ಹೆಚ್ಚಾಗಿ ReLU.

೧೧. Loss Function — ತಪ್ಪನ್ನು ಅಳೆಯುವುದು

ಇದು ಏನು?

Loss ಎಂದರೆ ನೆಟ್‌ವರ್ಕ್‌ನ ಉತ್ತರ ಎಷ್ಟು ತಪ್ಪು ಎಂದು ಹೇಳುವ ಒಂದೇ ಸಂಖ್ಯೆ. ಸಂಖ್ಯೆ ಚಿಕ್ಕದಷ್ಟೂ ಒಳ್ಳೆಯದು. ತರಬೇತಿಯ ಗುರಿ = loss ಕಡಿಮೆ ಮಾಡುವುದು.

(ಅ) Mean Squared Error — ಸಂಖ್ಯೆ ಊಹಿಸುವ ಕೆಲಸಕ್ಕೆ (ಬೆಲೆ, ತಾಪಮಾನ)

> **ಉದಾಹರಣೆ: ಮನೆ ಬಾಡಿಗೆ ಊಹೆ**
>
> ಮೂರು ಮನೆಗಳ ನಿಜವಾದ ಬಾಡಿಗೆ (ಸಾವಿರ ₹): 10, 15, 20. ನೆಟ್‌ವರ್ಕ್ ಊಹೆ: 12, 14, 26.
>
> ```
> ತಪ್ಪುಗಳು = (12−10)=2,  (14−15)=−1,  (26−20)=6
> ವರ್ಗ      = 4, 1, 36          (ವರ್ಗ ಮಾಡುವುದರಿಂದ ಋಣ ಚಿಹ್ನೆ ಹೋಗುತ್ತದೆ,
>                                 ದೊಡ್ಡ ತಪ್ಪಿಗೆ ದೊಡ್ಡ ಶಿಕ್ಷೆ)
> MSE       = (4 + 1 + 36) / 3 = 13.67
> ```

(ಆ) Cross-Entropy — ವರ್ಗೀಕರಣಕ್ಕೆ (ಬೆಕ್ಕು/ನಾಯಿ)

ಸರಿಯಾದ ವರ್ಗಕ್ಕೆ ನೆಟ್‌ವರ್ಕ್ ಕೊಟ್ಟ ಸಂಭವನೀಯತೆ *p* ಆದರೆ, loss = `−ln(p)`.

ಉದಾಹರಣೆ: ಚಿತ್ರ ಬೆಕ್ಕಿನದು (ಸತ್ಯ = ಬೆಕ್ಕು)

```
ನೆಟ್‌ವರ್ಕ್ 66% ಬೆಕ್ಕು ಎಂದರೆ (p = 0.659):  loss = −ln(0.659) = 0.417
ನೆಟ್‌ವರ್ಕ್ 99% ಬೆಕ್ಕು ಎಂದರೆ (p = 0.99) :  loss = −ln(0.99)  = 0.010   (ಬಹಳ ಚಿಕ್ಕದು)
ನೆಟ್‌ವರ್ಕ್ 5% ಬೆಕ್ಕು ಎಂದರೆ  (p = 0.05) :  loss = −ln(0.05)  = 3.00    (ದೊಡ್ಡ ಶಿಕ್ಷೆ)
```

ಆತ್ಮವಿಶ್ವಾಸದಿಂದ ತಪ್ಪು ಹೇಳಿದರೆ ಹೆಚ್ಚು ಶಿಕ್ಷೆ — ಇದೇ ಲಾಗರಿದಮ್‌ನ ಸೌಂದರ್ಯ.

೧೨. ಚೈನ್ ರೂಲ್ (Chain Rule) ಮತ್ತು Backpropagation

ಇದು ಏನು?

ನೆಟ್‌ವರ್ಕ್ ಒಂದು ಫಂಕ್ಷನ್ ಸರಪಳಿ: ತೂಕ → z → activation → loss. ಮೊದಲ ತೂಕ ಬದಲಾದರೆ loss ಮೇಲೆ ಆಗುವ ಪರಿಣಾಮ ತಿಳಿಯಲು, ಸರಪಳಿಯ ಪ್ರತಿ ಕೊಂಡಿಯ ವ್ಯುತ್ಪನ್ನವನ್ನು **ಗುಣಿಸುತ್ತಾ** ಹೋಗುತ್ತೇವೆ. ಇದೇ ಚೈನ್ ರೂಲ್. ಇದನ್ನು ಕೊನೆಯಿಂದ ಆರಂಭದ ಕಡೆಗೆ (ಹಿಂದಕ್ಕೆ) ಮಾಡುವುದಕ್ಕೆ **Backpropagation** ಎನ್ನುತ್ತಾರೆ.

ಚೈನ್ ರೂಲ್ ಸೂತ್ರ

$$\frac{dy}{dx} = \frac{dy}{du} \times \frac{du}{dx}$$

ಸರಳ ಉದಾಹರಣೆ: $y = (3x + 1)^2$, $x = 1$

$u = 3x + 1 = 4, \quad y = u^2 = 16$

$\frac{dy}{du} = 2u = 8, \quad \frac{du}{dx} = 3$

$\frac{dy}{dx} = 8 \times 3 = 24$

ನ್ಯೂರಾನ್‌ನ ಉದಾಹರಣೆ (Sigmoid + loss)

$x = 2$, $w = 0.5$, $b = 0.1$, ಗುರಿ $t = 1$. ಸರಪಳಿ: $z = w \cdot x + b \to a = \sigma(z) \to L = (a - t)^2$

ಮುಂದಕ್ಕೆ (forward):

$z = 0.5 \times 2 + 0.1 = 1.1$

$a = \sigma(1.1) = 0.750$

$L = (0.750 - 1)^2 = 0.0624$

ಹಿಂದಕ್ಕೆ (backward), ಪ್ರತಿ ಕೊಂಡಿಯ ವ್ಯುತ್ಪನ್ನ:

$\frac{dL}{da} = 2(a - t) = 2(0.750 - 1) = -0.499$

$\frac{da}{dz} = a(1 - a) = 0.750 \times 0.250 = 0.187$

$\frac{dz}{dw} = x = 2$

$\frac{dz}{db} = 1$

ಗುಣಿಸಿ:

$\frac{dL}{dw} = (-0.499) \times 0.187 \times 2 = -0.187$

$\frac{dL}{db} = (-0.499) \times 0.187 \times 1 = -0.094$

ಎರಡೂ ಋಣ → w ಮತ್ತು b ಹೆಚ್ಚಿಸಿದರೆ loss ಕಡಿಮೆ ಆಗುತ್ತದೆ.

೧೩. Gradient Descent — ಕಲಿಕೆಯ ಹೃದಯ

ಇದು ಏನು?

ಗ್ರೇಡಿಯಂಟ್ ತೋರಿಸುವ ದಿಕ್ಕಿನ **ವಿರುದ್ಧ** ಸಣ್ಣ ಹೆಜ್ಜೆ ಇಟ್ಟು ತೂಕ ಬದಲಿಸುವುದು:

```
w_ಹೊಸ = w_ಹಳೆಯ − (ಕಲಿಕಾ ದರ η) × (ಗ್ರೇಡಿಯಂಟ್)
```

ಕಲಿಕಾ ದರ (learning rate, η) ಎಂದರೆ ಹೆಜ್ಜೆಯ ಗಾತ್ರ. ತುಂಬ ದೊಡ್ಡದಾದರೆ ಕಣಿವೆಯನ್ನು ದಾಟಿ ಹಾರಿಹೋಗುತ್ತದೆ; ತುಂಬ ಚಿಕ್ಕದಾದರೆ ಕಲಿಕೆ ನಿಧಾನ.

ಉದಾಹರಣೆ ೧: ಒಂದೇ ತೂಕ, $L(w) = (w - 3)^2$ (ಉತ್ತಮ $w = 3$)

ಗ್ರೇಡಿಯಂಟ್ $= 2(w - 3)$. ಆರಂಭ $w = 0$, $\eta = 0.1$

ಹೆಜ್ಜೆ	w	ಗ್ರೇಡಿಯಂಟ್	ಹೊಸ w
1	0	$2(0-3) = -6$	$0 - 0.1\times(-6) = 0.6$
2	0.6	-4.8	1.08
3	1.08	-3.84	1.464

w ಹೆಜ್ಜೆ ಹೆಜ್ಜೆಯಾಗಿ 3 ರ ಕಡೆ ಹೋಗುತ್ತಿದೆ; ಗ್ರೇಡಿಯಂಟ್ ಚಿಕ್ಕದಾಗುತ್ತಿದೆ (ಹೆಜ್ಜೆಗಳು ತಾನಾಗಿ ಚಿಕ್ಕವಾಗುತ್ತವೆ).

೧೪. ಎಲ್ಲವನ್ನೂ ಸೇರಿಸಿ: ಒಂದು ನ್ಯೂರಾನ್‌ನ ಒಂದು ತರಬೇತಿ ಹಂತ

> **ಸಮಸ್ಯೆ**
>
> ಒಂದು ಇನ್‌ಪುಟ್ $x = 2$ ಕ್ಕೆ ಸರಿಯಾದ ಉತ್ತರ $t = 3$. ನಮ್ಮ ನ್ಯೂರಾನ್: $\hat{y} = w \cdot x + b$ (activation ಇಲ್ಲ). ಆರಂಭದಲ್ಲಿ $w = 0.5$, $b = 0.1$. ಕಲಿಕಾ ದರ $\eta = 0.05$. Loss $= (\hat{y} - t)^2$.

ಹಂತ	ಲೆಕ್ಕ	ಬಳಸಿದ ಗಣಿತ
೧. Forward	$\hat{y} = 0.5 \times 2 + 0.1 = \mathbf{1.1}$	ಬೀಜಗಣಿತ / ಡಾಟ್ ಪ್ರಾಡಕ್ಟ್
೨. Loss	$L = (1.1 - 3)^2 = (-1.9)^2 = \mathbf{3.61}$	ವರ್ಗ, MSE
೩. ಗ್ರೇಡಿಯಂಟ್	$dL/d\hat{y} = 2(\hat{y} - t) = -3.8$ $dL/dw = -3.8 \times x = \mathbf{-7.6}$ $dL/db = -3.8 \times 1 = \mathbf{-3.8}$	ವ್ಯುತ್ಪನ್ನ, ಚೈನ್ ರೂಲ್
೪. ಬದಲಾವಣೆ	$w = 0.5 - 0.05 \times (-7.6) = \mathbf{0.88}$ $b = 0.1 - 0.05 \times (-3.8) = \mathbf{0.29}$	Gradient Descent
೫. ಮತ್ತೆ ಪರೀಕ್ಷೆ	$\hat{y} = 0.88 \times 2 + 0.29 = \mathbf{2.05}$ $L = (2.05 - 3)^2 = \mathbf{0.9025}$	Forward ಪುನಃ

ಒಂದೇ ಹೆಜ್ಜೆಯಲ್ಲಿ ಊಹೆ 1.1 ರಿಂದ 2.05 ಕ್ಕೆ ಬಂತು ಮತ್ತು loss **3.61 ರಿಂದ 0.90** ಕ್ಕೆ ಇಳಿಯಿತು. ಇದೇ ಹಂತಗಳನ್ನು ಸಾವಿರಾರು ಬಾರಿ, ಸಾವಿರಾರು ತೂಕಗಳಿಗೆ ಪುನರಾವರ್ತಿಸಿದರೆ ಅದೇ "ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್ ತರಬೇತಿ".

> **ಒಂದೇ ವಾಕ್ಯದಲ್ಲಿ**
>
> ಊಹಿಸು (forward) → ತಪ್ಪು ಅಳೆ (loss) → ಯಾರು ಎಷ್ಟು ಕಾರಣ ಎಂದು ಕಂಡುಹಿಡಿ (backprop) → ಸ್ವಲ್ಪ ತಿದ್ದು (gradient descent) → ಮತ್ತೆ ಮತ್ತೆ ಮಾಡು.

ಇದನ್ನು Python ನಲ್ಲಿ ಓಡಿಸಿ ನೋಡಲು ಈ ರೆಪೊದಲ್ಲಿರುವ `python-examples/neural-networks/` ಮತ್ತು `python-examples/maths-ai/` ಫೋಲ್ಡರ್‌ಗಳ ಫೈಲ್‌ಗಳನ್ನು ನೋಡಿ.

೧೫. ಸಾರಾಂಶ ಕೋಷ್ಟಕ ಮತ್ತು ಕಲಿಕಾ ಪಟ್ಟಿ

ಗಣಿತ ವಿಷಯ	ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್‌ನಲ್ಲಿ ಬಳಕೆ	ಆದ್ಯತೆ
ಬೀಜಗಣಿತ ($y = wx + b$)	ನ್ಯೂರಾನ್‌ನ ಮೂಲ ಸೂತ್ರ	ಅತ್ಯಗತ್ಯ
ಸದಿಶ, ಡಾಟ್ ಪ್ರಾಡಕ್ಟ್	ಹಲವು ಇನ್‌ಪುಟ್ × ತೂಕ	ಅತ್ಯಗತ್ಯ
ಮ್ಯಾಟ್ರಿಕ್ಸ್ ಗುಣಾಕಾರ	ಒಂದು ಪದರದ ಎಲ್ಲ ನ್ಯೂರಾನ್‌ಗಳು ಒಟ್ಟಿಗೆ	ಅತ್ಯಗತ್ಯ
ಘಾತಾಂಕ ಮತ್ತು ಲಾಗ್	Sigmoid, Softmax, Cross-Entropy	ಅತ್ಯಗತ್ಯ
ಸಂಭವನೀಯತೆ	ವರ್ಗೀಕರಣದ ಔಟ್‌ಪುಟ್ ಅರ್ಥ	ಮುಖ್ಯ
ಅಂಕಿಅಂಶ	Normalization, ಮಾಹಿತಿ ಅರ್ಥಮಾಡಿಕೊಳ್ಳುವುದು	ಮುಖ್ಯ
ವ್ಯುತ್ಪನ್ನ	ಇಳಿಜಾರು → ಯಾವ ದಿಕ್ಕಿಗೆ ಬದಲಿಸಬೇಕು	ಅತ್ಯಗತ್ಯ
ಭಾಗಶಃ ವ್ಯುತ್ಪನ್ನ, ಗ್ರೇಡಿಯಂಟ್	ಸಾವಿರಾರು ತೂಕಗಳಿಗೆ ಒಟ್ಟಿಗೆ	ಅತ್ಯಗತ್ಯ
ಚೈನ್ ರೂಲ್	Backpropagation	ಅತ್ಯಗತ್ಯ
Gradient Descent	ತೂಕ ಬದಲಿಸುವ ವಿಧಾನ	ಅತ್ಯಗತ್ಯ

ಸ್ವಯಂ ಪರೀಕ್ಷೆ (ಉತ್ತರಗಳು ಕೆಳಗಿವೆ)

1. $x = [3, 1]$, $w = [2, -1]$, $b = 0.5$ ಆದರೆ z ಎಷ್ಟು?
2. $f(x) = x^2$ ಗೆ $x = 5$ ರಲ್ಲಿ ಇಳಿಜಾರು ಎಷ್ಟು?
3. $L(w) = (w - 4)^2$, $w = 1$, $\eta = 0.1$ ಆದರೆ ಒಂದು ಹೆಜ್ಜೆಯ ನಂತರ ಹೊಸ w ಎಷ್ಟು?
4. ಸರಿಯಾದ ವರ್ಗಕ್ಕೆ $p = 0.5$ ಕೊಟ್ಟರೆ cross-entropy loss ಎಷ್ಟು?

ಉತ್ತರಗಳು: (1) $3\times2 + 1\times(-1) + 0.5 = \mathbf{5.5}$ (2) $2\times5 = \mathbf{10}$ (3) ಗ್ರೇಡಿಯಂಟ್ $= 2(1-4) = -6$; $w = 1 - 0.1\times(-6) = \mathbf{1.6}$ (4) $-\ln(0.5) = \mathbf{0.693}$